Practical No. 5 Implement a decision tree algorithm and evaluate the performance.

Submitted By: Shantanu Yadav

Roll No.: 75

In [3]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

In [4]:
```
df =pd.read_csv('Cancer_Data11.csv')
df
```

Out[4]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mea
0	842302	M	17.99	10.38	122.80	1001.
1	842517	M	20.57	17.77	132.90	1326.
2	84300903	M	19.69	21.25	130.00	1203.
3	84348301	M	11.42	20.38	77.58	386.
4	84358402	M	20.29	14.34	135.10	1297.
...	...	...	...	...	...	...
564	926424	M	21.56	22.39	142.00	1479.
565	926682	M	20.13	28.25	131.20	1261.
566	926954	M	16.60	28.08	108.30	858.
567	927241	M	20.60	29.33	140.10	1265.
568	92751	B	7.76	24.54	47.92	181.

569 rows × 32 columns

In [5]:
```
df.shape
```

Out[5]:
```
(569, 32)
```

In [6]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   id                       569 non-null    int64  
 1   diagnosis                569 non-null    object 
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB
```

In [7]:
```
df.isnull().sum()
```

Out[7]:
```
id                        0
diagnosis                 0
radius_mean               0
texture_mean              0
perimeter_mean            0
area_mean                 0
smoothness_mean           0
compactness_mean          0
concavity_mean            0
concave points_mean       0
symmetry_mean             0
fractal_dimension_mean    0
radius_se                 0
texture_se                0
perimeter_se              0
area_se                   0
smoothness_se             0
compactness_se            0
concavity_se              0
concave points_se         0
symmetry_se               0
fractal_dimension_se      0
radius_worst              0
texture_worst             0
perimeter_worst           0
area_worst                0
smoothness_worst          0
compactness_worst         0
concavity_worst           0
concave points_worst      0
symmetry_worst            0
fractal_dimension_worst   0
dtype: int64
```

In [11]:
```
df.dtypes
```

Out[11]:
```
Sample_ID           object
Sequence            object
GC_Content         float64
AT_Content         float64
Sequence_Length      int64
Num_A                int64
Num_T                int64
Num_C                int64
Num_G                int64
kmer_3_freq        float64
Mutation_Flag        int64
Class_Label         object
Disease_Risk        object
dtype: object
```

In [10]:
```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df['diagnosis'] = le.fit_transform(df['diagnosis'])
```

In [11]:
```
df
```

Out[11]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mea
0	842302	1	17.99	10.38	122.80	1001.
1	842517	1	20.57	17.77	132.90	1326.
2	84300903	1	19.69	21.25	130.00	1203.
3	84348301	1	11.42	20.38	77.58	386.
4	84358402	1	20.29	14.34	135.10	1297.
...	...	...	...	...	...	.
564	926424	1	21.56	22.39	142.00	1479.
565	926682	1	20.13	28.25	131.20	1261.
566	926954	1	16.60	28.08	108.30	858.
567	927241	1	20.60	29.33	140.10	1265.
568	92751	0	7.76	24.54	47.92	181.

569 rows × 32 columns

In [12]:
```
X = df.drop('diagnosis',axis=1)
y = df['diagnosis']
```

In [13]:
```
X
```

Out[13]:

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothr
0	842302	17.99	10.38	122.80	1001.0	
1	842517	20.57	17.77	132.90	1326.0	
2	84300903	19.69	21.25	130.00	1203.0	
3	84348301	11.42	20.38	77.58	386.1	
4	84358402	20.29	14.34	135.10	1297.0	
...	...	...	...	...	...	
564	926424	21.56	22.39	142.00	1479.0	
565	926682	20.13	28.25	131.20	1261.0	
566	926954	16.60	28.08	108.30	858.1	
567	927241	20.60	29.33	140.10	1265.0	
568	92751	7.76	24.54	47.92	181.0	

569 rows × 31 columns

In [14]:
```
X.columns
```

Out[14]:
```
Index(['id', 'radius_mean', 'texture_mean', 'perimeter_mean', 'area_mea
n',
       'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness
_se',
       'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_
se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave points_worst',
       'symmetry_worst', 'fractal_dimension_worst'],
      dtype='object')
```

In [15]:
```
X
```

Out[15]:

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothr
0	842302	17.99	10.38	122.80	1001.0	
1	842517	20.57	17.77	132.90	1326.0	
2	84300903	19.69	21.25	130.00	1203.0	
3	84348301	11.42	20.38	77.58	386.1	
4	84358402	20.29	14.34	135.10	1297.0	
...	...	...	...	...	...	
564	926424	21.56	22.39	142.00	1479.0	
565	926682	20.13	28.25	131.20	1261.0	
566	926954	16.60	28.08	108.30	858.1	
567	927241	20.60	29.33	140.10	1265.0	
568	92751	7.76	24.54	47.92	181.0	

569 rows × 31 columns

In [16]:
```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confus
```

In [17]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
```

In [18]:
```
model = DecisionTreeClassifier(random_state=42)
```

In [19]:
```
model.fit(X_train, y_train)
```

Out[19]:

▾ DecisionTreeClassifier

▶ Parameters

In [20]:
```
y_pred = model.predict(X_test)
```

In [21]:
```
accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred)
confusion_mat = confusion_matrix(y_test, y_pred)
```

In [22]:
```
print("Accuracy:", accuracy)
print("Classification Report:\n", classification_rep)
```

```
Accuracy: 0.9385964912280702
Classification Report:
               precision    recall  f1-score   support

           0       0.96      0.94      0.95        71
           1       0.91      0.93      0.92        43

    accuracy                           0.94       114
   macro avg       0.93      0.94      0.93       114
weighted avg       0.94      0.94      0.94       114
```